

Selection Sort on Random, Reverse and Sorted Arrays

Aim

To implement the **Selection Sort algorithm in Python** and analyze its performance on **random, reverse-sorted, and already-sorted arrays**.

Algorithm

1. Start with an unsorted array.
2. Set the first position as the minimum element.
3. Compare the minimum element with all remaining elements.
4. Find the smallest element in the unsorted portion.
5. Swap the smallest element with the first element of the unsorted portion.
6. Move to the next position.
7. Repeat the process until the entire array is sorted.
8. Display the sorted arrays.

\text{Learn more}

Procedure

1. Open Python IDLE, VS Code, or any Python compiler.
2. Create three arrays:
 - Random array
 - Reverse-sorted array
 - Already-sorted array
3. Define a `selection_sort()` function.
4. In each pass, identify the smallest element from the unsorted part.
5. Swap it with the first element of the unsorted part.
6. Apply the function to all three arrays.
7. Display the original and sorted arrays.
8. Observe that Selection Sort performs the same basic number of comparisons even when the array is already sorted.

Python Code

```
def selection_sort(arr):
```

```
    n = len(arr)
```

```
    for i in range(n - 1):
```

```
        min_index = i
```

```

        for j in range(i + 1, n):
            if arr[j] < arr[min_index]:
                min_index = j

        arr[i], arr[min_index] = arr[min_index], arr[i]

# Random array
random_array = [64, 25, 12, 22, 11]

# Reverse-sorted array
reverse_array = [50, 40, 30, 20, 10]

# Already-sorted array
sorted_array = [10, 20, 30, 40, 50]

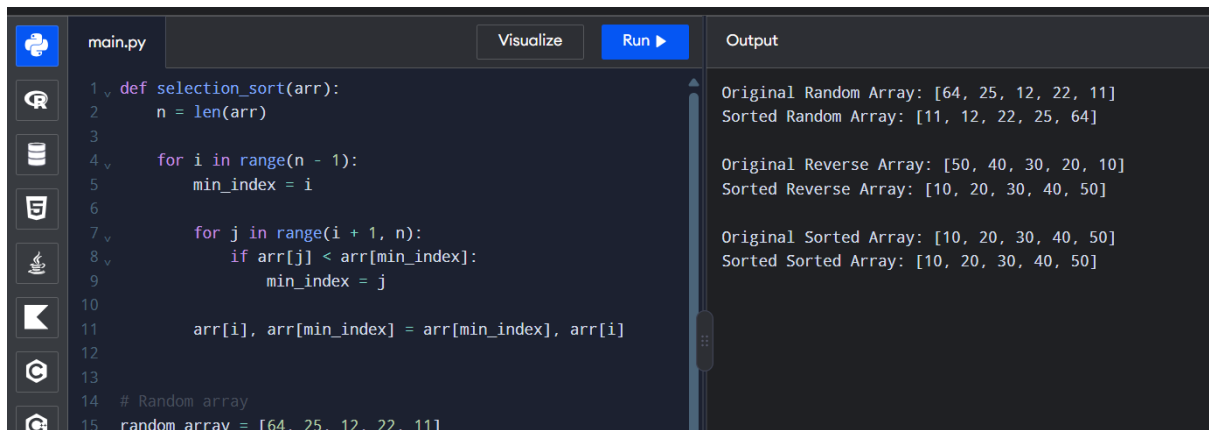
print("Original Random Array:", random_array)
selection_sort(random_array)
print("Sorted Random Array:", random_array)

print("\nOriginal Reverse Array:", reverse_array)
selection_sort(reverse_array)
print("Sorted Reverse Array:", reverse_array)

print("\nOriginal Sorted Array:", sorted_array)
selection_sort(sorted_array)
print("Sorted Sorted Array:", sorted_array)

```

Output



The screenshot shows a Python IDE with a file named 'main.py'. The code implements a selection sort algorithm. The output panel on the right displays the results of running the code on three different arrays: a random array, a reverse-sorted array, and an already-sorted array. In all cases, the output shows the original array followed by the sorted array in ascending order.

```
1 def selection_sort(arr):
2     n = len(arr)
3
4     for i in range(n - 1):
5         min_index = i
6
7         for j in range(i + 1, n):
8             if arr[j] < arr[min_index]:
9                 min_index = j
10
11         arr[i], arr[min_index] = arr[min_index], arr[i]
12
13
14 # Random array
15 random_array = [64, 25, 12, 22, 11]
```

Output

Original Random Array: [64, 25, 12, 22, 11]
Sorted Random Array: [11, 12, 22, 25, 64]

Original Reverse Array: [50, 40, 30, 20, 10]
Sorted Reverse Array: [10, 20, 30, 40, 50]

Original Sorted Array: [10, 20, 30, 40, 50]
Sorted Sorted Array: [10, 20, 30, 40, 50]

Result

The Selection Sort algorithm was successfully implemented in Python. The **random, reverse-sorted, and already-sorted arrays** were correctly sorted in ascending order. Selection Sort has a time complexity of **$O(n^2)$** for best, average, and worst cases.